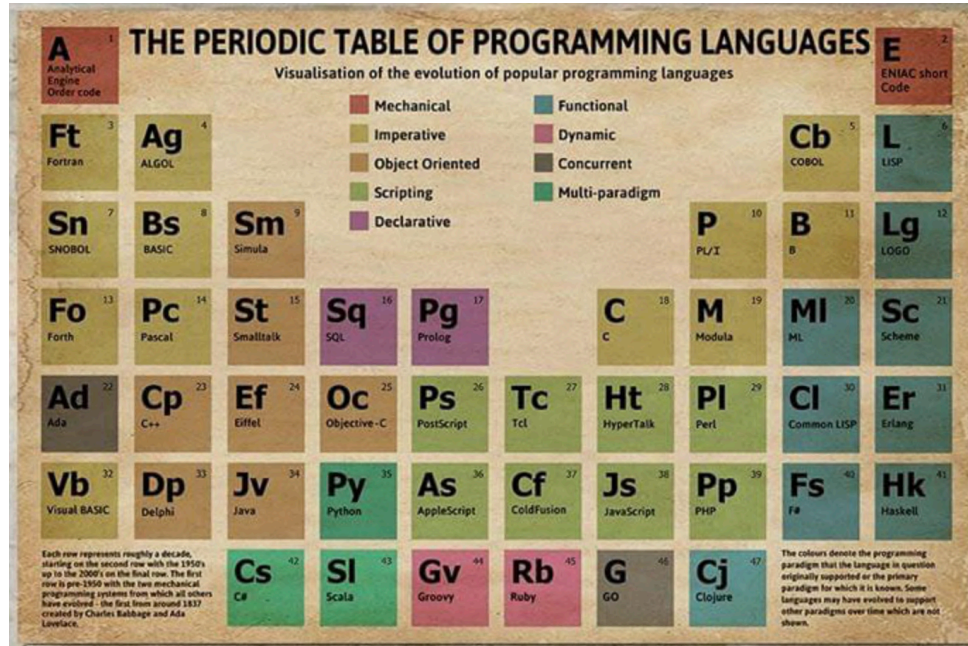


Llenguatges de Programació

Conceptes bàsics



Albert Rubio, Jordi Petit, Fernando Orejas, Gerard Escudero

UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Introducció

Un **llenguatge de programació** (LP) és un llenguatge formal utilitzat per controlar el comportament d'un computador tot implementant un algorisme.



Introducció

Tradicionalment, els LPs es consideren des de tres angles (anàlegs a la lingüística):

- **Sintaxi:** la forma dels programes correctes.
- **Semàntica:** el significat de les construccions dels programes.
- **Pragmàtica:** com s'usa el LP.

La majoria d'LPs venen definits per

- una especificació estàndard
ex: The C Programming Language, 1978
- una implementació de referència
ex: Standard ML

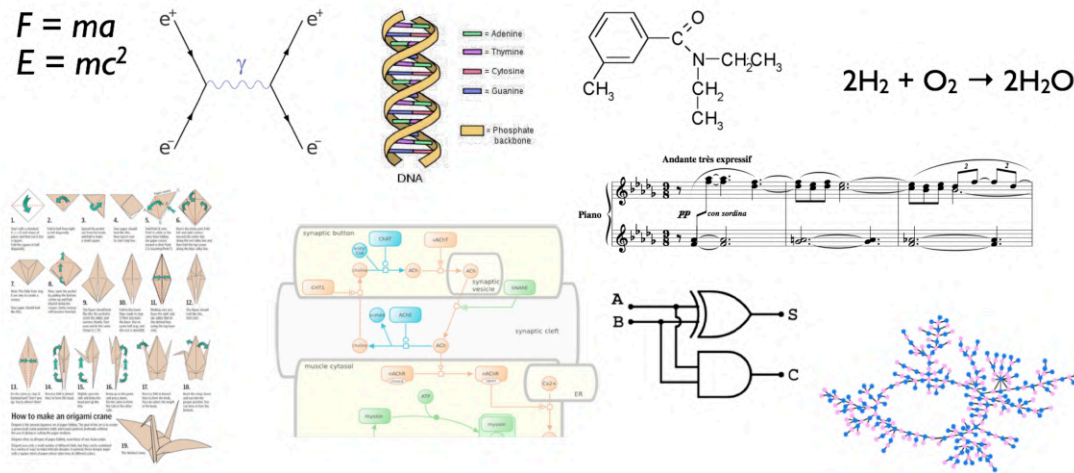
Perquè estudiar els LPs?

- **Visió enginyeril:** Necessitem LPs per programar computadors per fer coses xules.
- **Visió científica:** Necessitem LPs per descriure i raonar sobre aquestes computacions.

Perquè estudiar els LPs?

Els LPs, en tant que llenguatges, ens permeten representar conceptes i són una eina de pensament.

Exemples:



Perquè estudiar els LPs?

Els llenguatges que usem:

- influencien les nostres percepcions,
- guien i donen suport al nostre raonament,
- permeten i faciliten la nostra comunicació.

Exemple: Representacions dels nombres:

/ =

$$\overline{\text{VIII}}\text{CXCII} \div \text{MXXIV} = \text{VIII}$$

$$8192 / 1024 = 8$$

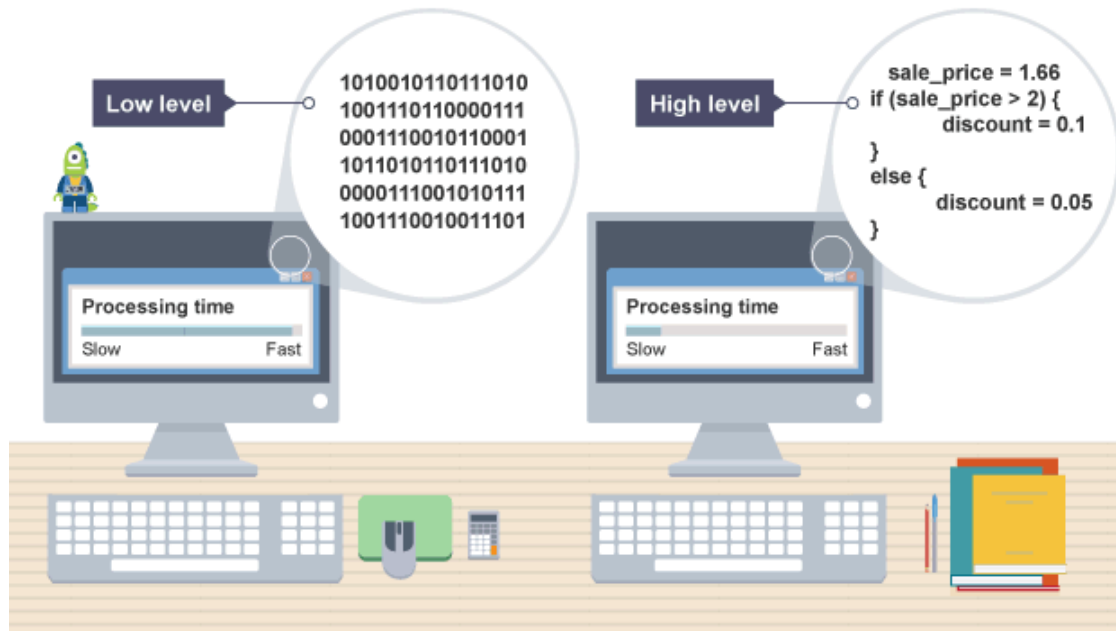
$$2^{13} / 2^{10} = 2^{13-10} = 2^3$$

Perquè estudiar els LPs?

Els llenguatges que usem:

- influencien les nostres percepcions,
- guien i donen suport al nostre raonament,
- permeten i faciliten la nostra comunicació.

Exemple: Codi màquina vs codi alt nivell:



Perquè estudiar els LPs?

Els llenguatges que usem:

- influencien les nostres percepcions,
- guien i donen suport al nostre raonament,
- permeten i faciliten la nostra comunicació.

Exemple: Programació imperativa vs declarativa:

```
let xs = [1, 2, 3, 4, 5]

let ys = []
for (var i = 0; i < xs.length; i++) {
  ys.push(xs[i] * 2);
}
```

```
let xs = [1, 2, 3, 4, 5]

let ys = xs.map(function (x,i) {
  return 2*x
})
```

Com estudiar els LPs?

Estudiar-los tots.



Estudiar les seves característiques,
els seus usos, avaluar les seves
qualitats, classificar-los en famílies,
conèixer la seva història, estudiar els
seus fonaments...



Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Característiques bàsiques d'un LP

- Tipus de dades: amb quines dades i objectes treballem.
- Sistema de tipus.
- Control de seqüència: en quin ordre s'executen les operacions.
- Control de dades. Com s'accedeix a les dades i als objectes.
- Entrada / Sortida.

Qualitats d'un LP

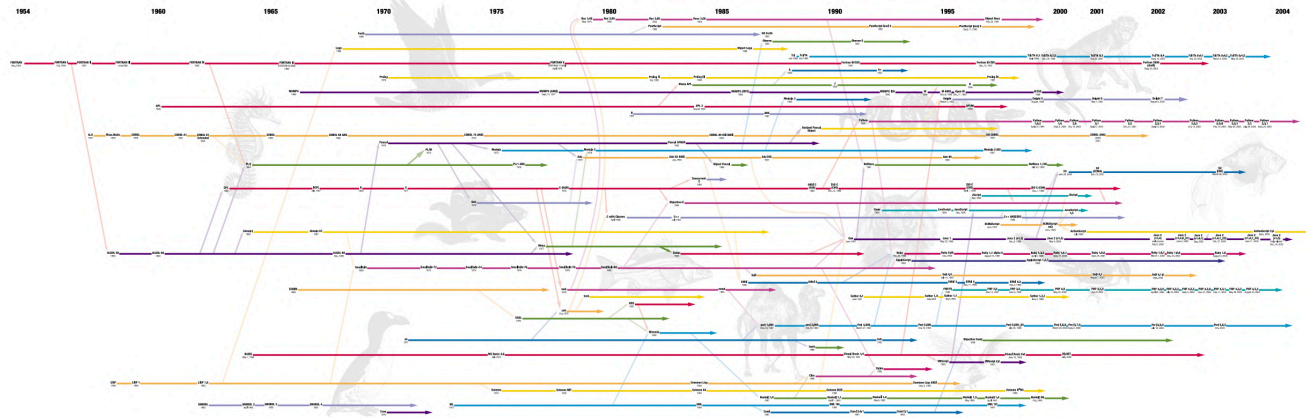
- Llegibilitat
- Eficiència
- Fiabilitat
- Expressivitat
- Simplicitat
- Nivell d'abstracció
- Adequació als problemes a tractar
- Facilitat d'ús
- Ortogonalitat

Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Història

📖 *History of Programming Languages*, O'Reilly



Història: Orígens

Tauletes babilòniques (2000ac)

A [rectangular] cistern.

The height is 3, 20, and a volume of 27, 46, 40 has been excavated. The length exceeds the width by 50. You should take the reciprocal of the height, 3, 20, obtaining 18. Multiply this by the volume, 27, 46, 40, obtaining 8, 20. Take half of 50 and square it, obtaining 10, 25. Add 8, 20, and you get 8, 30, 25. The square root is 2, 55. Make two copies of this, adding [25] to the one and subtracting from the other. You find that 3, 20 [i.e., $3 \frac{1}{3}$] is the length and 2, 30 [i.e., $2 \frac{1}{2}$] is the width.

This is the procedure.



📖 *Ancient Babylonian Algorithms*. D.E. Knuth, Communications ACM 1972.

🎬 *Math whizzes of ancient Babylon figured out forerunner of calculus*, YouTube.

Història: Orígens

Teler de Jacquard (1804)



Fotos: <http://www.revolutionfabrics.com/blog/2018/9/26/the-jacquard-loom-and-the-binary-code>

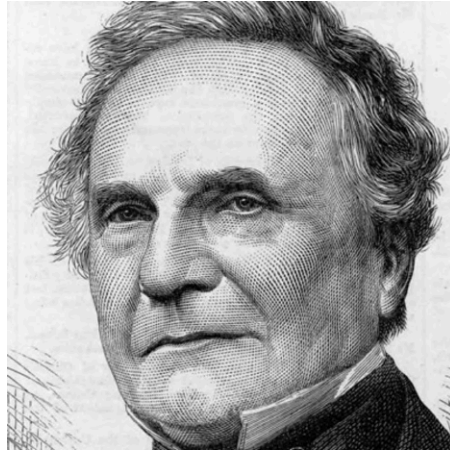
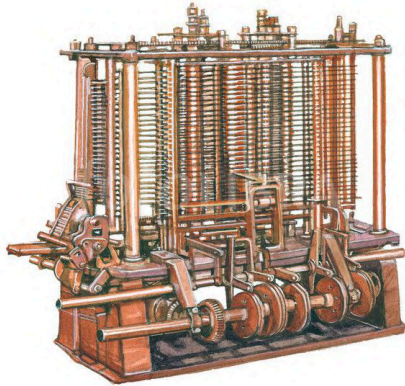


How an 1803 Jacquard Loom Lead to Computer Technology

Història: Orígens

Màquina analítica de Charles Babbage (1842)

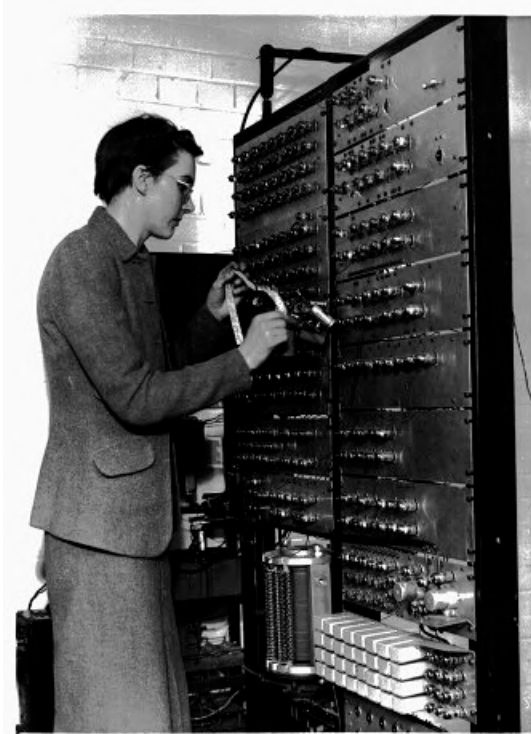
Es considera que Ada Lovelace és la primera programadora.



Fotos: Domini públic

 **Ada Lovelace, The World's First Computer Nerd?**

Història: Ensambladors



Kathleen Booth va escriure el primer llenguatge ensamblador (per un ordinador ARC al 1947).

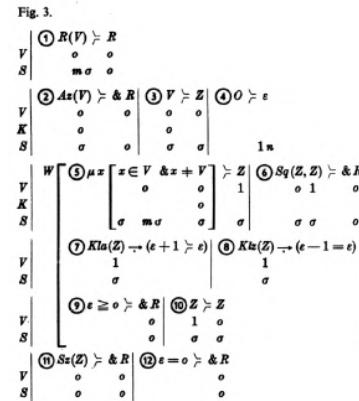
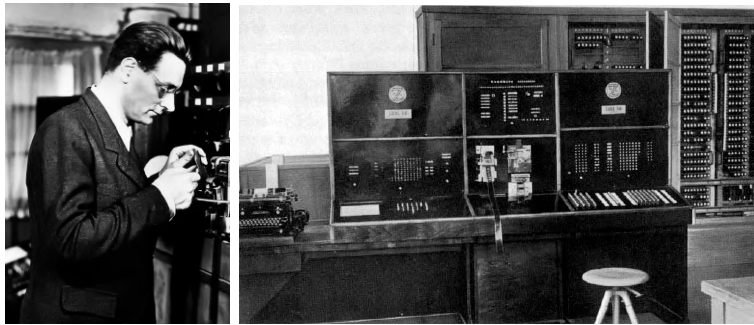
- 11) $M \times R \rightarrow cA$. Clear accumulator, multiply M by R and place L.H. 39 digits of answer in A and R.H. 39 digits in R.
- 12) $A \div M \rightarrow cR$. Clear register, divide A by M, leave quotient in R and remainder in A.
- 13) $C \rightarrow M_1$.
- 14) $C \rightarrow M_r$.
- 15) $Cc \rightarrow M_1$. If number in A ≥ 0 shift control to M_1 .
- 16) $Cc \rightarrow M_r$.
- 17) $A \rightarrow M$.

Fotos: Domini públic

📖 Kathleen Booth, Assembling Early Computers While Inventing Assembly

Història: Plankalkül

Primer LP d'alt nivell dissenyat per Konrad Zuse (1942-1945) pel seu ordinador a relés Z4. Implementat al 1990.



Fotos: Domini públic

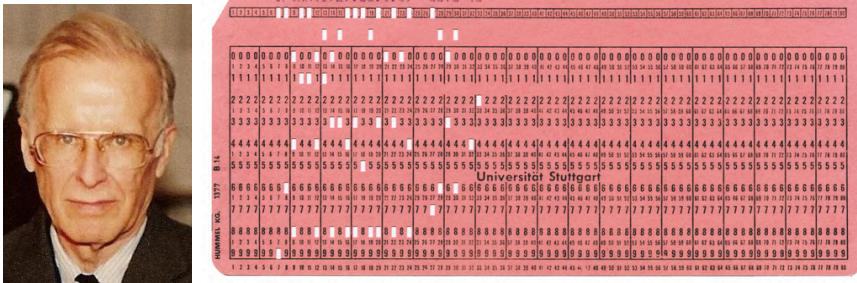
Història: Fortran

FORmula TRANslator (1954-1957). Desenvolupat per John Backus a IBM per a computació científica.

Es volia generar codi comparable al programat en ensamblador.

Idees principals:

- Variables amb noms (6 caràcters)
- Bucles i condicionals aritmètics
- E/S amb format
- Subrutines
- Taules



Fotos: Domini públic i Wikipedia

Història: Fortran

```
C AREA OF A TRIANGLE WITH A STANDARD SQUARE ROOT FUNCTION
      READ INPUT TAPE 5, 501, IA, IB, IC
      501 FORMAT (I5)
C IA, IB, AND IC MAY NOT BE NEGATIVE OR ZERO
C FURTHERMORE, THE SUM OF TWO SIDES OF A TRIANGLE
C MUST BE GREATER THAN THE THIRD SIDE, SO WE CHECK FOR THAT, TOO
      IF (IA) 777, 777, 701
      701 IF (IB) 777, 777, 702
      702 IF (IC) 777, 777, 703
      703 IF (IA+IB-IC) 777, 777, 704
      704 IF (IA+IC-IB) 777, 777, 705
      705 IF (IB+IC-IA) 777, 777, 799
      777 STOP 1
C USING HERON'S FORMULA WE CALCULATE THE
C AREA OF THE TRIANGLE
      799 S = FLOATF (IA + IB + IC) / 2.0
      AREA = SQRTF( S * (S - FLOATF(IA)) * (S - FLOATF(IB)) *
+      (S - FLOATF(IC)))
      WRITE OUTPUT TAPE 6, 601, IA, IB, IC, AREA
      601 FORMAT (4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
+      13H SQUARE UNITS)
      STOP
      END
```


Història: Fortran

THE FIRST PROGRAM

The first error message

The running of the first program, though well documented by Herb Bright [15], was not a planned activity. There is one error in Bright's report which needs correction -- 1957 April 20 was a Saturday not a Friday as reported. It was on the afternoon of that Friday when an unmarked deck of cards was delivered to Westinghouse-Bettis and which was assumed by Lew Ondis to be the right size to be a FORTRAN compiler. Jim Callaghan quickly wrote a small program based on a recent technical report by Ollie Swift, and using the "common" technique for running programs on the IBM 704, the unmarked deck was loaded into the syetm followed by the program. Surprisingly it worked and in a short time the first FORTRAN error message was output:

FORTRAN DIAGNOSTIC PROGRAM RESULTS

05065 SOURCE PROGRAM ERROR. THIS IS A TYPE-GO TO (),I
BUT THE RIGHT PARENTHESIS IS NOT FOLLOWED BY A COMMA

END OF DIAGNOSTIC PROGRAM RESULTS

The error was quickly fixed and the program (apparently) recompiled and executed to produce "... a whiff of computing followed by 28 pages of output ..."

Font: J.A.N. Lee, Twenty Five Years of Fortran

Història: COBOL

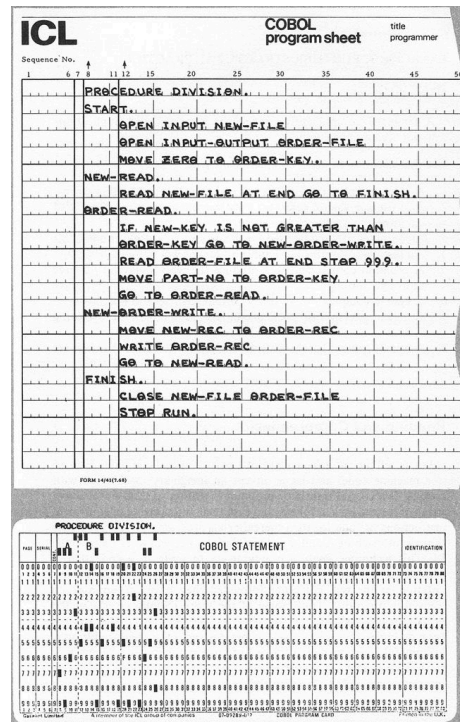
Common Business Oriented Language (1959). Desenvolupat per Grace Hopper pel DoD i fabricants per a aplicacions de gestió.

Idees principals:

- Vol semblar idioma anglès, sense símbols
- Macros
- Registres
- Fitxers
- Identificadors llargs (30 caràcters)



Foto: Domini públic



ICL COBOL program sheet title programmer

Sequence No. 1 6 7 8 11 12 15 20 25 30 35 40 45 50

PROCEDURE DIVISION.

START.

OPEN INPUT NEW-FILE

OPEN INPUT-OUTPUT ORDER-FILE

MOVE ZERO TO ORDER-KEY.

NEW-READ.

READ NEW-FILE AT END GO TO FINISH.

ORDER-READ.

IF NEW-KEY IS NOT GREATER THAN

ORDER-KEY GO TO NEW-ORDER-WRITE.

READ ORDER-FILE AT END STOP 999.

MOVE PART-NO TO ORDER-KEY

GO TO ORDER-READ.

NEW-ORDER-WRITE.

MOVE NEW-REC TO ORDER-REC

WRITE ORDER-REC

GO TO NEW-READ.

FINISH.

CLOSE NEW-FILE ORDER-FILE

STOP RUN.

FORM 1-4-1964-140

PROCEDURE DIVISION.

COBOL STATEMENT IDENTIFICATION

LINE	STATEMENT	IDENTIFICATION
1	START	
2	OPEN INPUT NEW-FILE	
3	OPEN INPUT-OUTPUT ORDER-FILE	
4	MOVE ZERO TO ORDER-KEY.	
5	NEW-READ.	
6	READ NEW-FILE AT END GO TO FINISH.	
7	ORDER-READ.	
8	IF NEW-KEY IS NOT GREATER THAN	
9	ORDER-KEY GO TO NEW-ORDER-WRITE.	
10	READ ORDER-FILE AT END STOP 999.	
11	MOVE PART-NO TO ORDER-KEY	
12	GO TO ORDER-READ.	
13	NEW-ORDER-WRITE.	
14	MOVE NEW-REC TO ORDER-REC	
15	WRITE ORDER-REC	
16	GO TO NEW-READ.	
17	FINISH.	
18	CLOSE NEW-FILE ORDER-FILE	
19	STOP RUN.	

FORM 1-4-1964-140

Història: COBOL

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. Multiplier.  
AUTHOR. Michael Coughlan.  
* Example program using ACCEPT, DISPLAY and MULTIPLY to  
* get two single digit numbers from the user and multiply them together  
  
DATA DIVISION.  
  
WORKING-STORAGE SECTION.  
01 Num1 PIC 9 VALUE ZEROS.  
01 Num2 PIC 9 VALUE ZEROS.  
01 Result PIC 99 VALUE ZEROS.  
  
PROCEDURE DIVISION.  
    DISPLAY "Enter first number (1 digit) : " WITH NO ADVANCING.  
    ACCEPT Num1.  
    DISPLAY "Enter second number (1 digit) : " WITH NO ADVANCING.  
    ACCEPT Num2.  
    MULTIPLY Num1 BY Num2 GIVING Result.  
    DISPLAY "Result is = ", Result.  
    STOP RUN.
```

Història: LISP

LIS^t Processing (1958). Desenvolupat per John McCarthy al MIT per a recerca en IA.

Idees principals:

- Sintaxi uniforme
- Funcions (composició i recursivitat)
- Llistes
- Expressions simbòliques
- Recol·lector de brossa



Foto: Wikipedia

Història: LISP

```
(defun factorial (N)
  "Compute the factorial of N."
  (if (= N 1)
      1
      (* N (factorial (- N 1)))))
```

```
(defun first-name (name)
  "Select the first name from a name represented as a list."
  (first name))

(setf names '((John Q Public) (Malcolm X)
              (Admiral Grace Murray Hopper) (Spot)
              (Aristotle) (A A Milne) (Z Z Top)
              (Sir Larry Olivier) (Miss Scarlet)))
```

Història: Algol

ALGOrithmic Language (1958). Dissenyat com un llenguatge universal per computació científica. No gaire popular, però dóna lloc a LPs com Pascal, C, C++, and Java.

Idees principals:

- Blocs amb àmbits de variables
- Pas per valor i pas per nom ($\neq$ pas per referència)
- Recursivitat
- Gramàtica formal (Backus-Naur Form or BNF)

```
procedure Absmax(a) Size:(n, m) Result:(y) Subscripts:(i, k);  
  value n, m; array a; integer n, m, i, k; real y;  
begin  
  integer p, q;  
  y := 0; i := k := 1;  
  for p := 1 step 1 until n do  
    for q := 1 step 1 until m do  
      if abs(a[p, q]) > y then  
        begin y := abs(a[p, q]);  
              i := p; k := q  
        end  
      end  
    end  
  end  
end Absmax
```

Història: Algol

Pas per nom:

```
begin
  integer i;
  integer array A[0:9];

  procedure ini (x); integer x; (* pas per nom *)
  begin
    for i := 0 step 1 until 9 do x := 0
  end;

  ini(A[i]);
  ...
end
```

El paràmetre `x` es passa per nom (amb `value x`; es passaria per valor).

Dins de `ini()`, aparentment, s'assigna 10 cops 0 a `x`.

Però `x` és el "nom" del paràmetre real, és a dir `A[i]`.

Per tant, `ini()` realment inicialitza a zero tot l'array.

Exercici

Què fa aquest programa?

```
begin
  integer i, sum;
  integer array A[0:9];

  procedure suma (x, s); integer x, s;
  begin
    s := 0;
    for i := 0 step 1 until 9 do s := s + x
  end;

  suma(A[i], sum);
end
```

Història: altres llenguatges

- Basic (Orientat a l'ensenyament de la programació) - 1964
- Pascal/Algol 68 (els hereus directes d'Algol 60) - 1970/1968
- C (Definit per programar Unix) - 1972
- Prolog (Primer llenguatge de programació lògica) - 1972
- Simula 67/Smalltalk 80 (primers llenguatges OO)
- Ada (LP creat per ser utilitzat pel Departament de Defensa Americà) - 1980
- ML/Miranda (primers llenguatges funcionals moderns) - 1983/1986
- C++ (llenguatge dinàmic i flexible compatible amb C) - 1983
- Java (llenguatge orientat a objectes per a torradores) - 1990
- Python (llenguatge llegible d'alt nivell de propòsit general) - 1991
- ...

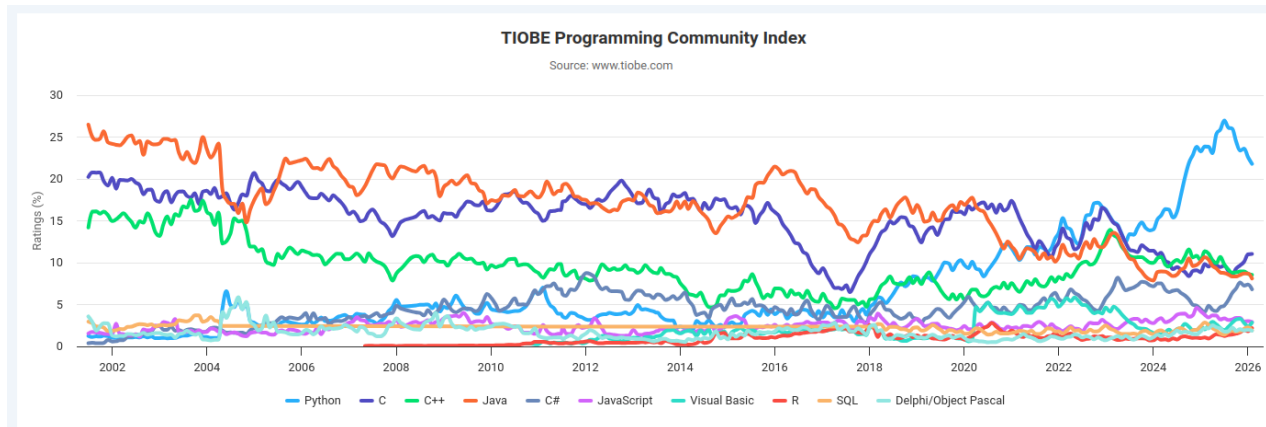
Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Ús dels LPs

Com mesurar la popularitat dels LPs?

- TIOBE Programming Community Index

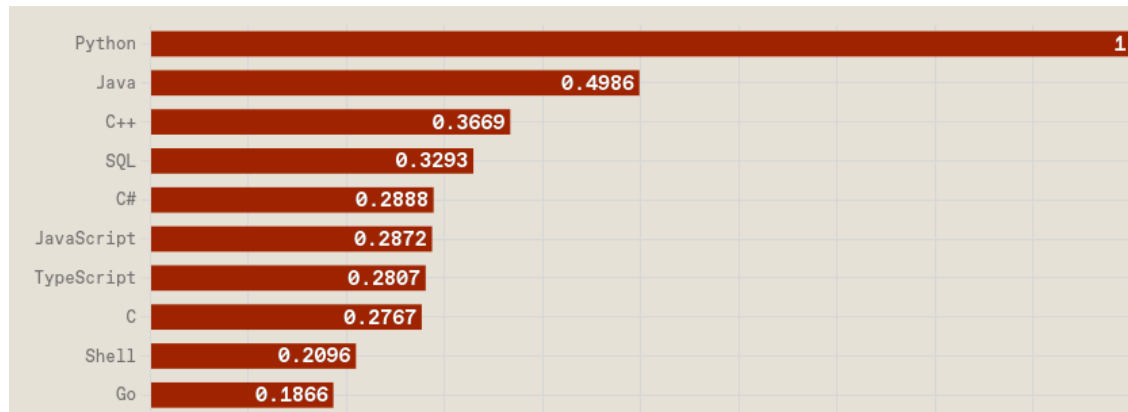


Font: <https://www.tiobe.com/tiobe-index/> (2026)

Ús dels LPs

Com mesurar la popularitat dels LPs?

- IEEE Spectrum ranking: The Top Programming Languages 2025

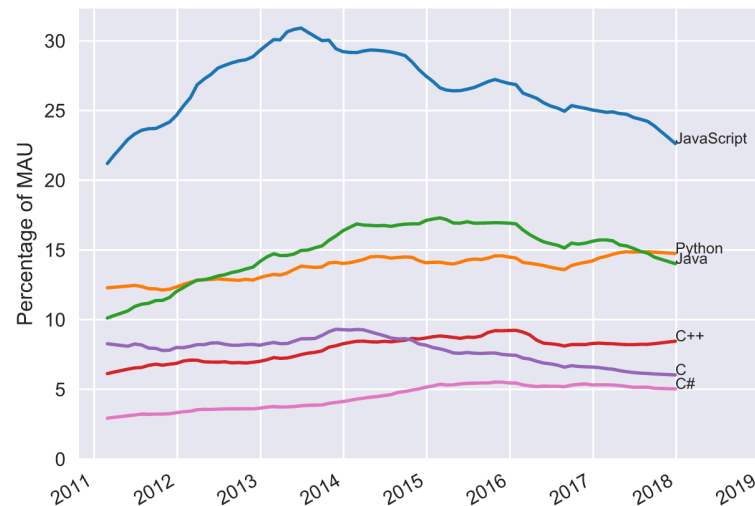
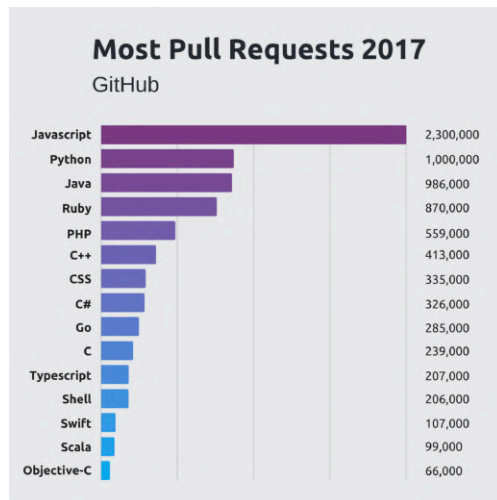


Font: <https://spectrum.ieee.org/top-programming-languages-2025>

Ús dels LPs

Com mesurar la popularitat dels LPs?

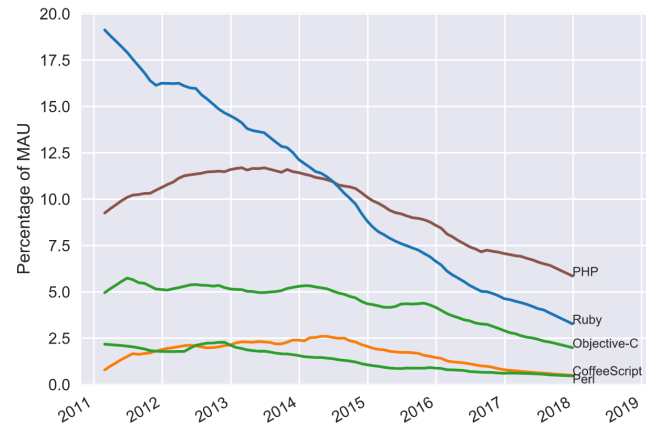
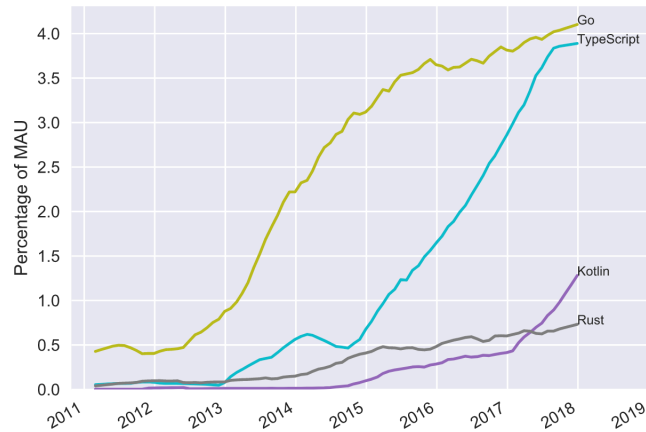
- Estadístiques de GitHub



Fonts: <https://github.com> i <https://www.benfrederickson.com/ranking-programming-languages-by-github-users/>

Ús dels LPs

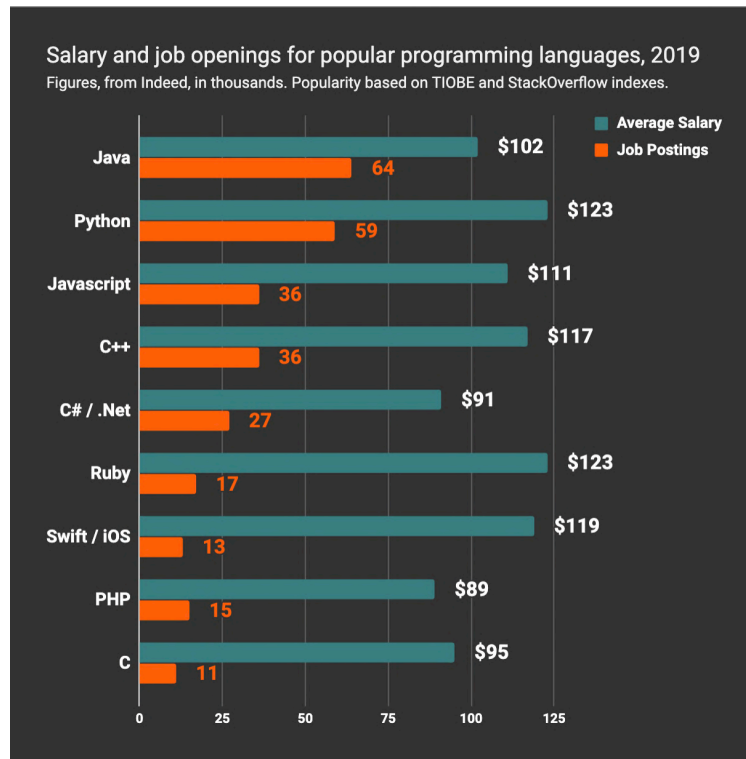
Quins LPs estudiar/evitar?



Font: <https://www.benfrederickson.com/ranking-programming-languages-by-github-users/>

Ús dels LPs

Quins LPs estudiar/evitar?



Font: <https://www.codeplatoon.org/the-best-paying-and-most-in-demand-programming-languages-in-2019/>

Ús dels LPs (1965 - 2025)



Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Paradigmes de LPs

Els paradigmes de programació classifiquen els LPs segons les seves característiques.

The Paradigms of Programming

Robert W. Floyd
Stanford University



Paradigm(pæ·radim, -dəim) . . . [a. F. *paradigme*, ad. L. *paradigma*, a. Gr. *παράδειγμα* pattern, example, f. *παράδεικνυ·ναι* to exhibit beside, show side by side. . .]
1. A pattern, exemplar, example.

1752 J. Gill *Trinity* v. 91

The archetype, paradigm, exemplar, and idea,
according to which all things were made.

From the Oxford English Dictionary.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.
Author's address: Department of Computer Science, Stanford University, Stanford CA 94305.
© 1979 ACM 0001-0782/79/0800-0455 \$00.75.

455

Today I want to talk about the paradigms of programming, how they affect our success as designers of computer programs, how they should be taught, and how they should be embodied in our programming languages.

A familiar example of a paradigm of programming is the technique of *structured programming*, which appears to be the dominant paradigm in most current treatments of programming methodology. Structured programming, as formulated by Dijkstra [6], Wirth [27, 29], and Parnas [21], among others, consists of two phases.

In the first phase, that of top-down design, or stepwise refinement, the problem is decomposed into a very small number of simpler subproblems. In programming the solution of simultaneous linear equations, say, the first level of decomposition would be into a stage of triangularizing the equations and a following stage of back-substitution in the triangularized system. This gradual decomposition is continued until the subproblems that arise are simple enough to cope with directly. In the simultaneous equation example, the back substitution process would be further decomposed as a backwards iteration of a process which finds and stores the value of the *i*th variable from the *i*th equation. Yet further decomposition would yield a fully detailed algorithm.

The second phase of the structured programming paradigm entails working upward from the concrete objects and functions of the underlying machine to the more abstract objects and functions used throughout the modules produced by the top-down design. In the linear equation example, if the coefficients of the equations are rational functions of one variable, we might first design

Communications
of
the ACM

August 1979
Volume 22
Number 8

 *The paradigms of programming.* R. Floyd, Communications ACM 1979.

Paradigmes de LPs

Paradigmes comuns:

- **Imperatiu:** Les instruccions precisen els canvis d'estat.

```
f = 1;  
while (--n) f *= n;
```

- **Declaratiu:** Es caracteritza el resultat, però no com calcular-lo.

```
select full_name, order_date, order_amount  
from customers inner join orders  
on customers.customer_id = orders.customer_id
```

- **Funcional:** El resultat és el valor d'una sèrie d'aplicacions de funcions.

```
cincMesGrans :: [a] -> [a]  
cincMesGrans = take 5 . reverse . sort
```

Paradigma imperatiu

Característiques:

- Noció d'estat
- Instruccions per canviar l'estat
- Efectes laterals

Exemples:

- C/C++, Python, Java, Ensamblador, ...

Útils quan, per exemple, l'eficiència és clau.

Paradigma imperatiu

Subclassificacions:

- **Procedural:** Les instruccions s'agrupen en procediments.

```
PROCEDURE swap (VAR a, b: INTEGER)
VAR c: INTEGER;
BEGIN
    c := a;  b := a;  a := c;
END;
```

- **Orientat a objectes:** Les instruccions s'agrupen amb l'estat dels objectes sobre les quals operen.

```
Point»dist: aPoint
    dx := aPoint x - x.
    dy := aPoint y - y.
    ↑ ((dx * dx) + (dy * dy)) sqrt
```

Paradigma declaratiu

Característiques:

- Llenguatges descriptius.
- El programa diu què s'ha de fer, però no necessàriament com.

Utilitat:

- Prototipat d'aplicacions amb forta component simbòlica, problemes combinatoris, etc.
- Consultes en bases de dades relacionals o lògiques.
- Per especificació i raonament automàtic.

Exemples:

- SQL (consultes relacionals) / GraphQL (consultes en grafs, amb tipus) / Prolog (lògica de primer ordre) / Matemàtiques

Paradigma declaratiu

SQL: Trobar tots els emails dels usuaris del Jutge amb el Hello World acceptat.

```
EXPLAIN
SELECT DISTINCT email
FROM Users JOIN Submissions USING (user_id)
WHERE problem_id='P68688_en' AND veredict='AC'
ORDER BY email
```

Planificador (postgres):

QUERY PLAN

```
-----
Unique  (cost=56242.20..56290.03 rows=9566 width=30)
  -> Sort  (cost=56242.20..56266.11 rows=9566 width=30)
      Sort Key: users.email
      -> Hash Join  (cost=2889.46..55609.71 rows=9566 width=30)
          Hash Cond: (submissions.user_id = users.user_id)
          -> Bitmap Heap Scan on submissions  (cost=570.63..53265.77 rows=95
              Recheck Cond: (problem_id = 'P68688_en'::text)
              Filter: (veredict = 'AC'::text)
              -> Bitmap Index Scan on idx_submissions_problem_id  (cost=0.
                  Index Cond: (problem_id = 'P68688_en'::text)
              -> Hash  (cost=1860.59..1860.59 rows=36659 width=37)
                  -> Seq Scan on users  (cost=0.00..1860.59 rows=36659 width=3
```

Paradigma declaratiu

Subclasificacions:

- **Consultes:** Resposta a consultes a una base de dades.

```
SELECT full_name, order_date, order_amount  
FROM customers INNER JOIN orders  
ON customers.customer_id = orders.customer_id
```

- **Matemàtic:** El resultat es declara com a la solució d'un problema d'optimització.

```
Maximize  
  x1 + 2 x2 + 3 x3 + x4  
Subject To  
  - x1 + x2 + x3 + 10 x4 ≤ 20  
  x1 - 3 x2 + x3 ≤ 30  
  x2 - 3.5 x4 = 0
```

- **Lògic:** Resposta a una pregunta amb fets i regles.

```
human(socrates).  
mortal(X) :- human(X).  
?- mortal(socrates).
```

Paradigma funcional

Característiques:

- Procedural
- Sense noció d'estat i sense efectes laterals
- Més fàcil de raonar (sobre correctesa o sobre transformacions)

Utilitat:

- Útils per al prototipat, fases inicials de desenvolupament (especificacions executables i transformables).
- Tractament simbòlic.
- Sistemes de tipus potents (incloent polimorfisme paramètric i inferència de tipus).

Exemples: Haskell, ML (Caml, OCaml), Erlang, XSLT (tractament XML),...

Paradigma funcional

Conceptes clau:

- **Recursivitat:**

```
(define (fib n)
  (cond
    ((= n 0) 0)
    ((= n 1) 1)
    (else
     (+ (fib (- n 1))
        (fib (- n 2))))))
```

- **Funcions d'ordre superior:** funcions que reben o retornen funcions.

```
map(lambda x: 2*x, [1,2,3,4])
```

- **Aplicació parcial (currying):**

```
double = (*2)
```


Paradigma funcional

Conceptes clau:

- **Funcions pures:** Amb els mateixos arguments, les funcions sempre retornen el mateix resultat. No hi ha efectes laterals.
- **Mecanismes d'avaluació:** Avaluació estricta *vs* avaluació mandrosa.
- Sistemes de tipus.

Paradigma 00

Característiques:

- Es basa en *objectes* (atributs + mètodes) i potser *classes*.
- Inclou principalment *polimorfisme (subtipat)* i *herència*.
- Poden tenir sistemes de tipus complexos.

Exemples: Smalltalk, Simula, C++, Java...

Llenguatges multiparadigma

Molts LPs combinen diferents paradigmes. Per exemple:

- Python, Perl: **imperatiu** + orientat a objectes + funcional
- OCaml: **funcional** + imperatiu + orientat a objectes

Alguns LPs incorporen característiques d'altres paradigmes:

- Prolog: **lògic** (+ imperatiu + funcional)

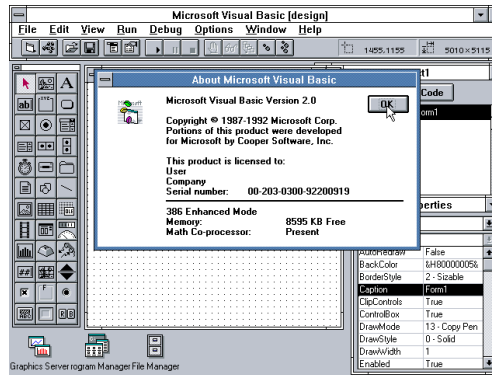
Altres combinacions:

- Erlang: **funcional** + concurrent + distribuït

Llenguatges visuals

En un **llenguatge de programació visual** (VPL) els programes són creats manipulant elements gràfics.

- **Visual Basic:** Dialecte de BASIC utilitzant una interfície gràfica per facilitar la creació d'interfícies gràfiques (1991).



- **Scratch:** Eina d'educació per a nens a partir de 8 anys (2003).

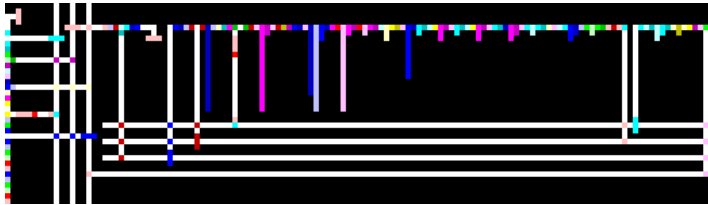


Llenguatges esotèrics

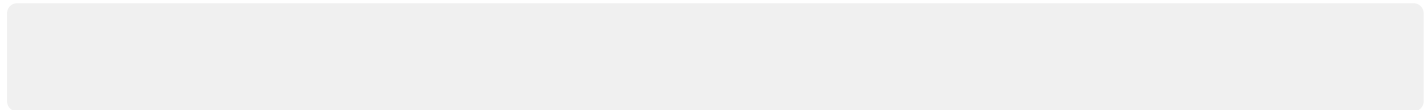
- **Brainfuck:** Basat en màquines de Turing, només té 8 instruccions. Hello world:

```
,>+++++[<----->-], [<+>-], <.>.
```

- **Piet:** Usa imatges amb 20 colors com a codi font. Factorial:



- **Whitespace:** Només té espais en blanc i tabuladors. BFS:



(indenteu-lo bé quan el copieu 😊)

Llenguatges esotèrics

- **Shakespeare:** Amaga un programa dins d'una obra de teatre.

The Infamous Hello World Program.

Romeo, a young man with a remarkable patience.

Juliet, a likewise young woman of remarkable grace.

Ophelia, a remarkable woman much in dispute with Hamlet.

Hamlet, the flatterer of Andersen Insulting A/S.

Act I: Hamlet's insults and flattery.

Scene I: The insulting of Romeo.

[Enter Hamlet and Romeo]

Hamlet:

You lying stupid fatherless big smelly half-witted coward!

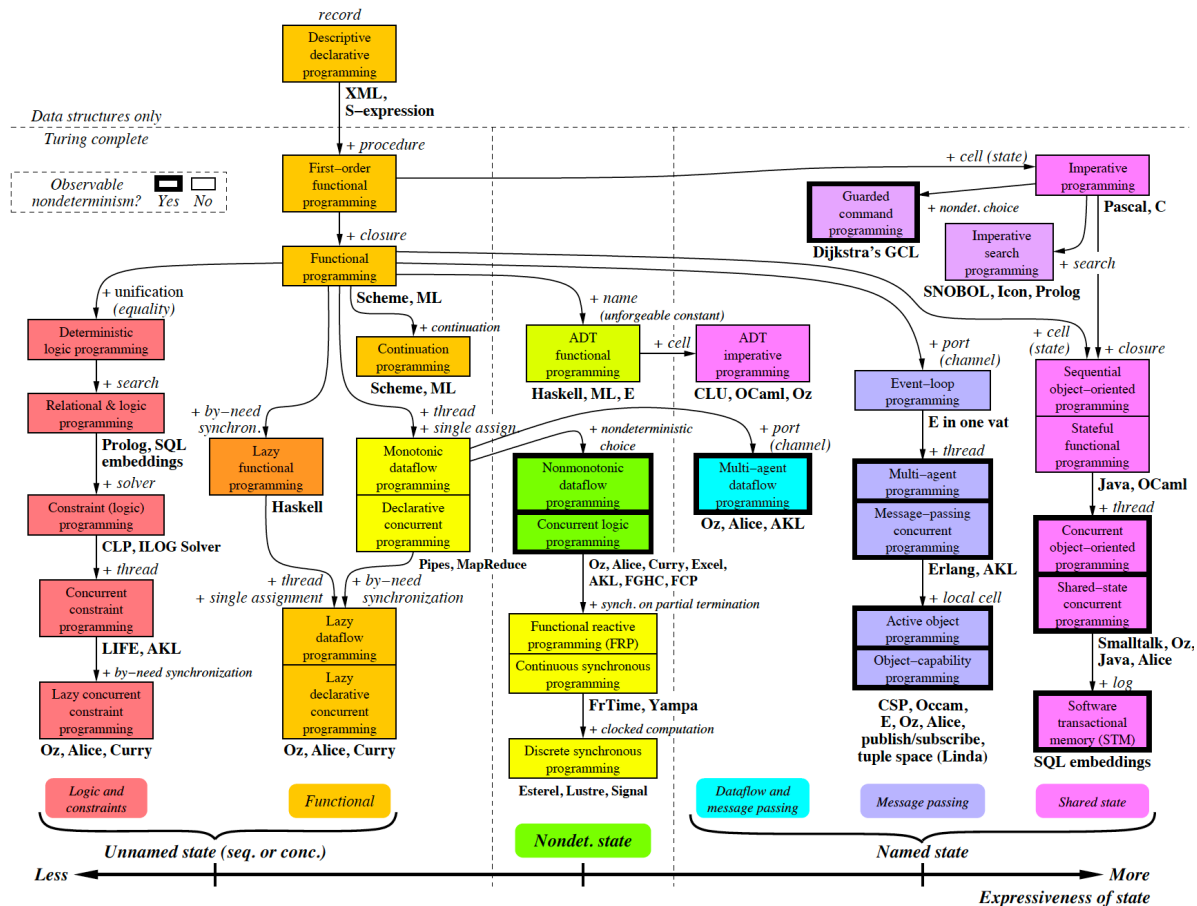
You are as stupid as the difference between a handsome rich brave
hero and thyself! Speak your mind!

You are as brave as the sum of your fat little stuffed misused dusty
old rotten codpiece and a beautiful fair warm peaceful sunny summer's
day. You are as healthy as the difference between the sum of the
sweetest reddest rose and my father and yourself! Speak your mind!

You are as cowardly as the sum of yourself and the difference
between a big mighty proud kingdom and a horse. Speak your mind.

Speak your mind!

Paradigmes de LPs: Més taxonomies



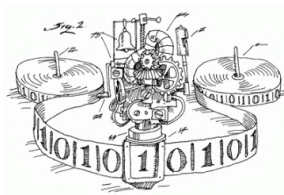
Font: *Programming Paradigms for Dummies*, P. Van Roy

Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Turing completa

Màquina de Turing: model matemàtic de càlcul imperatiu molt simple (Alan Turing, 1936)



- Cinta infinita amb un capçal mòvil per llegir/escriure símbols
- Conjunt finit d'estats
- Funció de transició (estat, símbol $\rightarrow$ estat, símbol, moviment)

λ -càlcul: model matemàtic de càlcul funcional molt simple. (Alonzo Church, 1936).

$(\lambda (y) x (yz)) (ab)$
 $\downarrow$
 $x(abz)$



- Sistema de reescriptura
- Basat en abstracció i aplicació de funcions

Tesi de Church-Turing: "Tot algorisme és computable amb una Màquina de Turing o amb una funció en λ -càlcul".

Fotos: Wikipedia, Fair Use, [Lambda Calculus for Absolute Dummies](#)

Turing completa

Un LP és **Turing complet** si pot implementar qualsevol càlcul que un computador digital pugui realitzar.

Alguns autors consideren només com a LPs els llenguatges Turing complets.

- LPs Turing complets:
 - LPs de programació de propòsit general (C/C++, Python, Haskell...)
 - automàts cel·lulars (Joc de la vida, ...)
 - alguns jocs (Minecraft, Buscamines...)

Per ser Turing complet només cal tenir salts condicionals (bàsicament, `if` i `goto`) i memòria arbitràriament gran.

- LPs no Turing complets:
 - Expressions regulars (a Perl o a AWK)
 - ANSI SQL

Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Sistemes d'execució

- **Compilat:** el codi és transforma en codi objecte i després es monta en un executable. Sol ser eficient. Es distribueix l'executable.

Exemples: C, C++, Ada, Haskell, ...

- **Interpretat:** el codi s'executa directament o el codi es transforma en codi d'una màquina virtual, que l'executa. Es distribueix el codi font.

Aumenten la portabilitat i l'expressibilitat (es poden fer més coses en temps d'execució) però disminueix l'eficiència.

Exemples: Python, JavaScript, Prolog (WAM), Java (JVM), ...

Sistemes mixtes:

- **Just in Time compilation:** Es compila (parcialment) en temps d'execució.
- Alguns interpretats, poden ser també compilats (per exemple, Prolog).
- I al revés (Haskell).

⇒ El sistema d'execució depèn més de la implementació que del LP.

Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Sistemes de tipus

Un **sistema de tipus** és un conjunt de regles que assignen *tipus* als elements d'un programa (com ara variables, expressions, funcions...) per evitar errors.

La comprovació de tipus verifica que les diferents parts d'un programa es comuniquin adequadament en funció dels seus tipus.

Per exemple, amb:

```
class Persona;  
class Forma;  
class Rectangle :Forma;  
class Triangle :Forma;  
  
double area (const Forma&);
```

- Cridar a `area` amb un `Rectangle` o un `Triangle` és correcte,
- Cridar a `area` amb una `Persona` o un enter és un error de tipus.

Errors de tipus

Un **error de tipus** consisteix en aplicar a les dades una operació que el seu tipus no suporta.

- Type Cast: usar un valor d'un tipus en un altre tipus.
 - 👉 En C, un enter pot ser usat com a funció (com a adreça), però pot saltar on no hi ha una funció i pot provocant un error.
 - 👉 En C, `printf("%s", 42);`.
- Aritmètica de punters.
 - 👉 En C++, si tenim `A p[10];` llavors `p[15]` té tipus `A`, però el que hi ha a `p[15]` pot ser qualsevol altra cosa i pot provocar un error de tipus.
- Alliberament explícit de memòria (`deallocate/delete`).

Exemple: En Pascal usar un apuntador alliberat pot donar errors de tipus.
- En llenguatges OO (antics), no existència d'un mètode (degut a l'herència).

Seguretat de tipus

La **seguretat de tipus** (*type safety*) és la mesura de com de fàcil/difícil és cometre errors de tipus en un LP.

Exemples:

- C té reputació de ser un LP sense gaire seguretat de tipus (unions, enumerats, `void*`, ...).
- C++ hereta C però proporciona més seguretat de tipus: els enumerats no es poden convertir implícitament amb els enters o altres enumerats, el *dynamic casting* pot comprovar errors de tipus en temps execució, ...
- Java és dissenyat per a proporcionar seguretat de tipus. Però es pot abusar del *classloader*.
- Python, igual que Java, està dissenyat per donar seguretat de tipus, malgrat que el tipatge sigui dinàmic.
- Es creu que Haskell és *type safe* si no s'abusa d'algunes construccions com `unsafePerformIO`.

Tipat fort vs feble

Tipat fort (*strong typing*): L'LP imposa restriccions que eviten barrejar valors de diferents tipus (conversions explícites).

Tipat feble (*weak typing*): L'LP té regles màgiques per convertir tipus automàticament.

Javascript

```
4 + '7';    // '47'  
4 * '7';    // '28'
```

PHP

```
4 + '7';    // '11'  
4 * '7';    // '28'
```

Python

```
4 + '7'      #  TypeError: unsupported operand type(s) for +: 'int' and 'str'  
4 * '7'      # '7777'
```



ShadowCheetah @shadowcheets · 12 d'ag. de 2019



Javascript is weird.

```
> ('b' + 'a' + + 'a' + 'a').toLowerCase()  
< "banana"
```



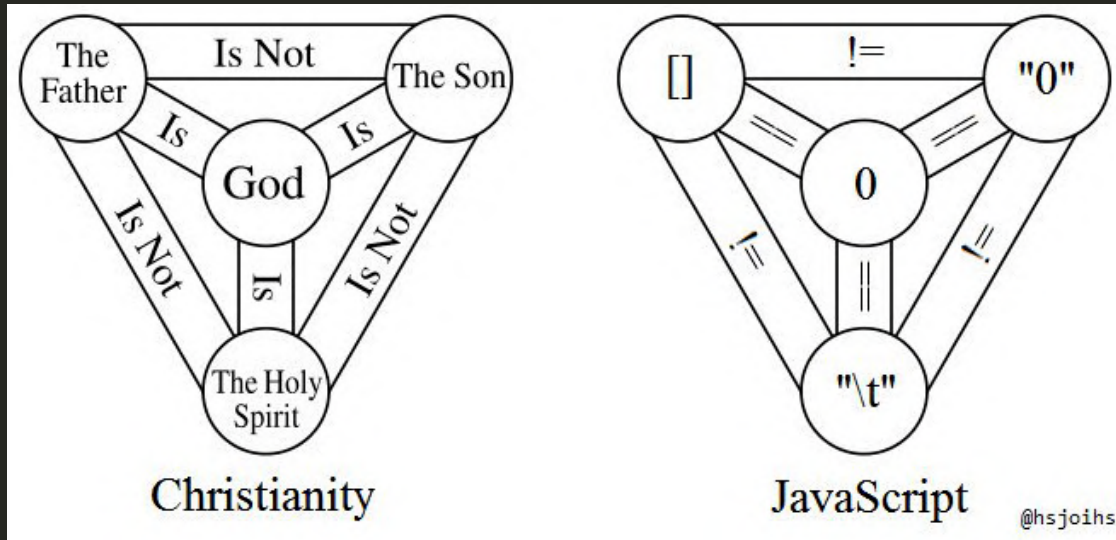
```
jpetit@bach (node) %$1
jpetit@bach ~ > node
Welcome to Node.js v15.8.0.
Type ".help" for more information.
> 0 > null
false
> 0 >= null
true
> 0 == null
false
> 0 <= null
true
> 0 < null
false
> 
```

Arrays equal Strings?!

04 May 2018

```
var a = [1,2,3];  
var b = [1,2,3];  
var c = '1,2,3';
```

```
a == c; // true  
b == c; // true  
a == b; // false
```



<https://javascriptwtf.com/>

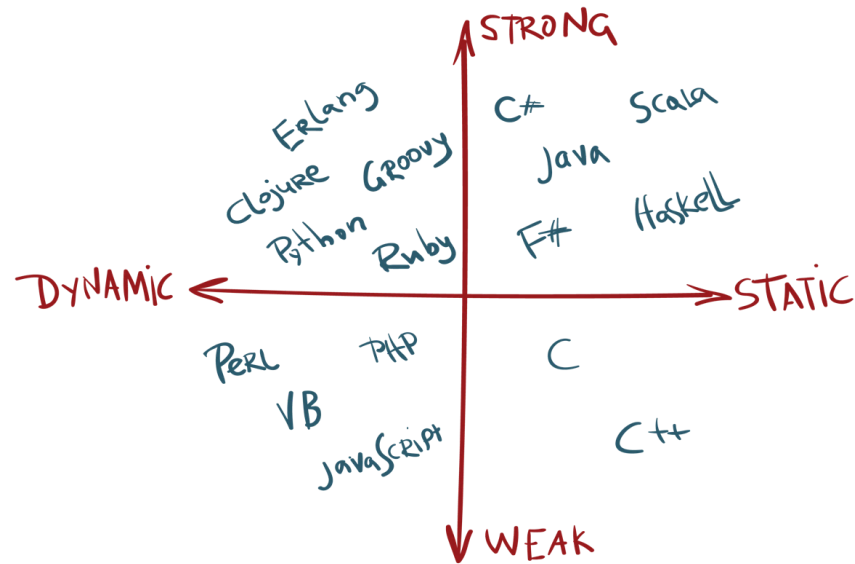
<https://wtfjs.com/>

Comprovació de tipus

La comprovació de tipus pot ser:

- **Estàtica:** en temps de compilació.
- **Dinàmica:** en temps d'execució.
- **Mixta.**

Sistemas de tipos



Font

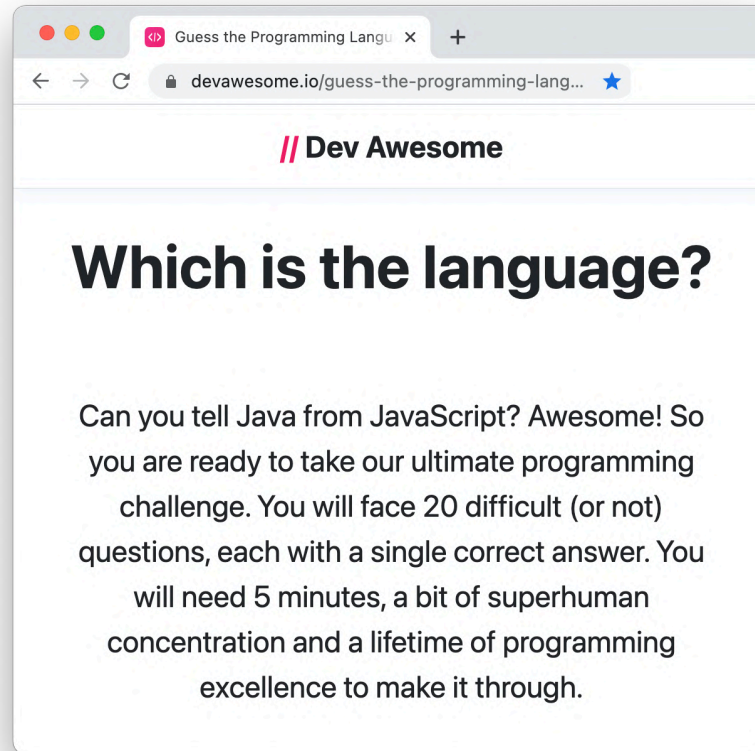
Contingut

- Introducció
- Característiques
- Història
- Ús dels LPs
- Paradigmes
- Turing completeness
- Sistemes d'execució
- Sistemes de tipus
- Exercicis

Exercicis

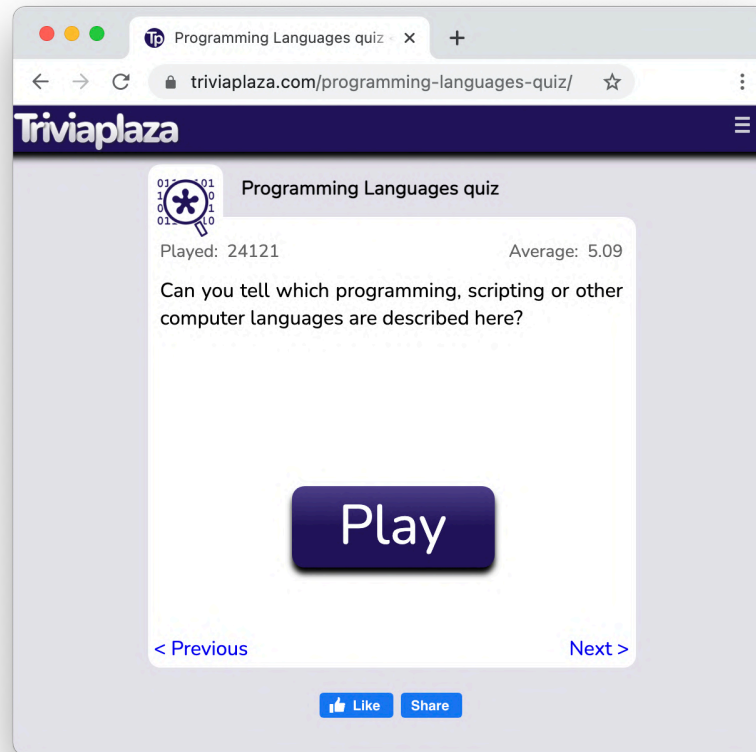
1. Digueu quin és el vostre LP favorit.
2. Expliqueu quines són les seves propietats fonamentals:
 - Propòsit del llenguatge.
 - Paradigma o paradigmes de programació que admet el llenguatge.
 - Sistema d'execució.
 - Sistema de tipus.
 - Principals aplicacions.
 - Història del LP i relació amb LPs similars.
 - Exemples de codi il·lustratiu.
 - Altres característiques particulars.

Exercicis



<https://devawesome.io/guess-the-programming-language/>

Exercicis



<https://www.triviaplaza.com/programming-languages-quiz/>